Project Proposal

Team Name: OnboardBuddies
Project / App Name: OnboardBuddy
Team Members:
Nam Le - DevOps
Bradley - Backend
Sahib - Backend (+ UI/UX)
Eugene - Frontend (+ QA)

Overview:
OnboardBuddy is an AI-powered web dashboard that analyzes a repository (code + git history + tests/coverage) and generates an onboarding/handoff plan, including architecture maps, class dependency graphs, and interactive walkthrough tutorials so developers can become productive without hours of manual code reading or repeated explanations.

1) Project Summary

1.1 Project Description

OnboardBuddy analyses a repository and produces:

- **System architecture map** (services/modules/components and how they connect)
- **Class dependency graph** (file/class/function-level relationships)
- **Interactive tutorials** (step-by-step walkthroughs of key workflows)
- A **dashboard + generated files** (markdown, diagrams, summaries) that can be updated over time (optionally via GitHub)

Core analysis uses **AST extraction** (starting with TypeScript), plus overlays such as:

- **Test coverage** (to identify core functionality and confidence areas)
- **Docs/README/comments** signals (to identify what's documented vs missing)

1.2 Inspirations

- **Claude Code / similar tools** that re-scrape codebases each run (pain point: repetitive analysis with no retained "memory")
- General dev onboarding pain: returning to a project or handing off knowledge costs hours

Borrowed elements (conceptually):

- "Repo -> summary -> next actions" workflow (but extended into diagrams + tutorials)
- Emphasis on local analysis for privacy (especially for proprietary code)

1.3 Target Users

- **Developers** joining a team or codebase for the first time
- **Interns / new hires** needing guided onboarding
- **Maintainers** returning to a project after time away
- **Teams doing handoffs** (e.g., feature teams rotating ownership)

1.4 Data (Collected / Used / Stored)

Inputs (collected/used locally):

- Repository contents (source code, configs, docs)
- Git metadata (commit history, file churn, authorship counts—configurable)
- Test results and coverage reports

Stored outputs (in app DB and/or generated files):

- Parsed metadata (AST summaries, symbol indexes, dependency edges)
- Derived metrics (hotspots, core modules, coverage hotspots)
- Generated documentation artifacts (markdown, tutorial steps, diagrams)
- User account data (email/username, hashed passwords or OAuth tokens)

Privacy note: in the first parts of the project we will use Codex or Deepseek, so we can't promise privacy, but in the future phases as a stretch goal we want to include privacy (default mode - public mode, or switch in setting to use private mode - local LLM model).

1.5 Non-trivial Features

1. **Multi-language AST ingestion pipeline (starting with TypeScript)**

    - Requires correct parsing, symbol extraction, and resilient handling of real-world code patterns.
2. **Architecture + class dependency graph generation (classes/functions/modules)**
    - Non-trivial to infer meaningful relationships (imports, call graph, data flow) without overwhelming noise.
3. **Critical-path identification using overlays (git churn + coverage + docs)**
    - Requires combining signals to rank “the 25% you need first.”
4. **Interactive tutorial generator**
    - Must convert static analysis into a usable “do this, then this” walkthrough with code pointers.
5. **Incremental re-analysis (update instead of full re-scrape)**
    - Uses git diffs to update indexes/graphs efficiently.

1.6 Standard Features

- **User authentication** (login/signup, sessions/JWT)
- **GitHub Repo registration/import** (local path selection + project profile)
- **Dashboard pages** (graphs, docs, tutorials)
- **CRUD for projects** (create project, rescan, delete project, join project/add team)
- **Basic admin/settings** (LLM settings, privacy settings, export settings)
- **File storage / export** (download zip of generated docs/diagrams)

1.7 Risks (Data / Privacy / Scope)

Privacy & security risks

- Repos may contain proprietary code, secrets, credentials, PII.
    - Mitigation: zero data retention with local-first processing (in final phases try to use local models like Ollama), encryption-at-rest for stored artifacts, opt-in cloud model usage. SOC2 and DPA if needed.
- Storing Git metadata could reveal team identities/behavior.
    - Mitigation: allow disabling author analysis; anonymize authors.

Scope risks

- “Full architecture understanding” across languages is large.
    - Mitigation: start with TypeScript only, with a plugin architecture for adding languages later.
- Data flow + call graphs can become very complex.
    - Mitigation: start with module-level + key entrypoints; allow drill-down.

LLM risks

- Hallucinated explanations or incorrect tutorials.
    - Mitigation: always link explanations to exact code locations and confidence levels; use citations to files/lines; provide “verify” steps.

2) Core Technical Requirements

2.1 Non-trivial Core Tasks

- **Repo ingestion & indexing**
  - Read repo tree, detect language/tooling, build file indexes.
- **TypeScript AST pipeline**
  - Parse TS/TSX using TypeScript compiler API.
  - Extract: modules, imports/exports, functions, classes, interfaces, routes/handlers patterns.
- **Class dependency graph engine**
  - Build edges: import graph, symbol reference graph, call graph (best-effort).
  - Store as graph data (nodes/edges) for visualization.
- **Hotspot analysis via Git history (Stretch)**
  - Compute churn metrics: commits touching file, lines changed, recent activity window.
- **Test coverage overlay**
  - Ingest coverage report; map to file paths; annotate graphs with coverage %.
- **Architecture map & "critical paths"**
  - Identify entry points (server start, routing, major services).
  - Rank "most important 25%" using weighted scoring (coverage + churn + centrality).
- **Tutorial generator**
  - Produce step-by-step walkthroughs:
    - Build/deploy flow
    - Auth flow
    - Database write flow
    - Request lifecycle (Data flows)

2.2 Standard Features

- **Auth** (JWT or session cookies)
- **Projects CRUD**
- **Dashboard**
- **GitHub integration**
- **Settings** (privacy / LLM provider / model selection)
- **File storage / export** (download zip of generated docs/diagrams)
- **Basic logging** and error reporting (local)

2.3 Stretch Goals

- **Local analysis through CLI**
- **Git change frequency** (to identify "hot" files)
- **MCP server integration**
- **Local-first models usage**

3) Project Team & Task Assignment

Team Member	Role	Main Responsibilities	Why
Eugene	Frontend / QA	Dashboard UI, graph views (D3/React Flow), tutorial UX, exports, project settings pages	Visual clarity determines product value
Bradley	Backend	Repo ingestion API, TS AST parsing, dependency graph generation, persistence schema	Core engine logic
Sahib	Backend / UI/UX	Git history analysis, coverage ingestion, scoring/ranking, tutorial generation pipeline, testing	Critical-path identification + quality
Nam	DevOps / DB	Docker setup, MongoDB schema setup, CI/CD, deployment pipeline, security baseline	Reliability and repeatable builds

4) Tech Stack & Architecture

4.1 Tech Stack

Frontend

- React (MERN)
- Graph visualization: React Flow / D3.js (TBD)
- Auth UI (Supabase) + project dashboard pages

Backend

- Node.js + Express
- TypeScript
- Analysis engine modules (AST parsing, graph builder, scoring)

Database

- MongoDB (projects, analysis snapshots, artifacts metadata, user accounts)

CLI (Stretch)

- Node-based CLI (invokes local analysis + optionally syncs results to server)

Infra

- Docker + Docker Compose for local dev (app + DB)
- CI/CD (GitHub Actions)
- Deployment (Railway)

LLM Integration

- Start with closed source, then optionally local Ollama open source models

4.2 System Architecture Map (High-level)

Components

1. **Web Dashboard (React)**
    - Project creation and management
    - Visualization: architecture map + class dependency graphs + coverage/hotspots overlays (stretch)
    - Tutorial player (step-by-step onboarding)
    - Export engine (download generated docs/diagrams)
2. **Backend API (Node/Express, TypeScript)**
    - Authentication + user/org/project permissions
    - Project repo connection settings (e.g., GitHub URL, branch, optional token)
    - Triggers analysis runs and serves the latest snapshot/graph data to the dashboard
3. **Analysis Engine (Server-side service/module)**
    - Repo fetch/sync (GitHub clone/pull, or uploaded zip—pick one approach)
    - TypeScript AST parsing + symbol indexing
    - Dependency extraction (imports + symbol references + best-effort call graph)
    - Overlay modules:
        - Git churn/hotspot analysis
        - Test coverage ingestion + mapping
        - Docs quality signals (README, comments, JSDoc, ADRs)
    - Ranking/scoring to identify the “top 25%” code areas/ critical paths
    - Tutorial + documentation generator (LLM-assisted)
    - Generates analysis bundle (JSON graphs + markdown + references)
4. **Persistence Layer (MongoDB)**
    - Stores projects, analysis snapshots, graphs, tutorial steps, and exports
    - Snapshot versioning: “latest” + history to track architectural drift

Typical Flow (how pieces connect)

1. User logs in -> creates a **Project**
2. User connects a repo (GitHub URL + branch) **or uploads repo snapshot**
3. User clicks **“Run Analysis”**
4. Backend enqueues analysis job -> Analysis Engine:
    - syncs repo -> builds AST index -> builds graphs -> overlays churn/coverage -> ranks critical paths -> generates tutorial/docs
5. Results stored as a **Snapshot**
6. Dashboard reads Snapshot -> renders maps, graphs, tutorials -> user exports docs

4.3 Critical Code Paths

1. **Repo acquisition** (clone/pull or upload) -> file index
2. **AST parse -> symbol/index extraction** (TypeScript first)
3. **Dependency graph generation** (module + class/function relationships)
4. **Coverage import + mapping** (lcov -> files)
5. **Git churn metrics** (**Stretch**; hot files, recent activity)
6. **Ranking/scoring** -> “Top 25% to read first”
7. **Tutorial/doc generation** -> dashboard rendering + export

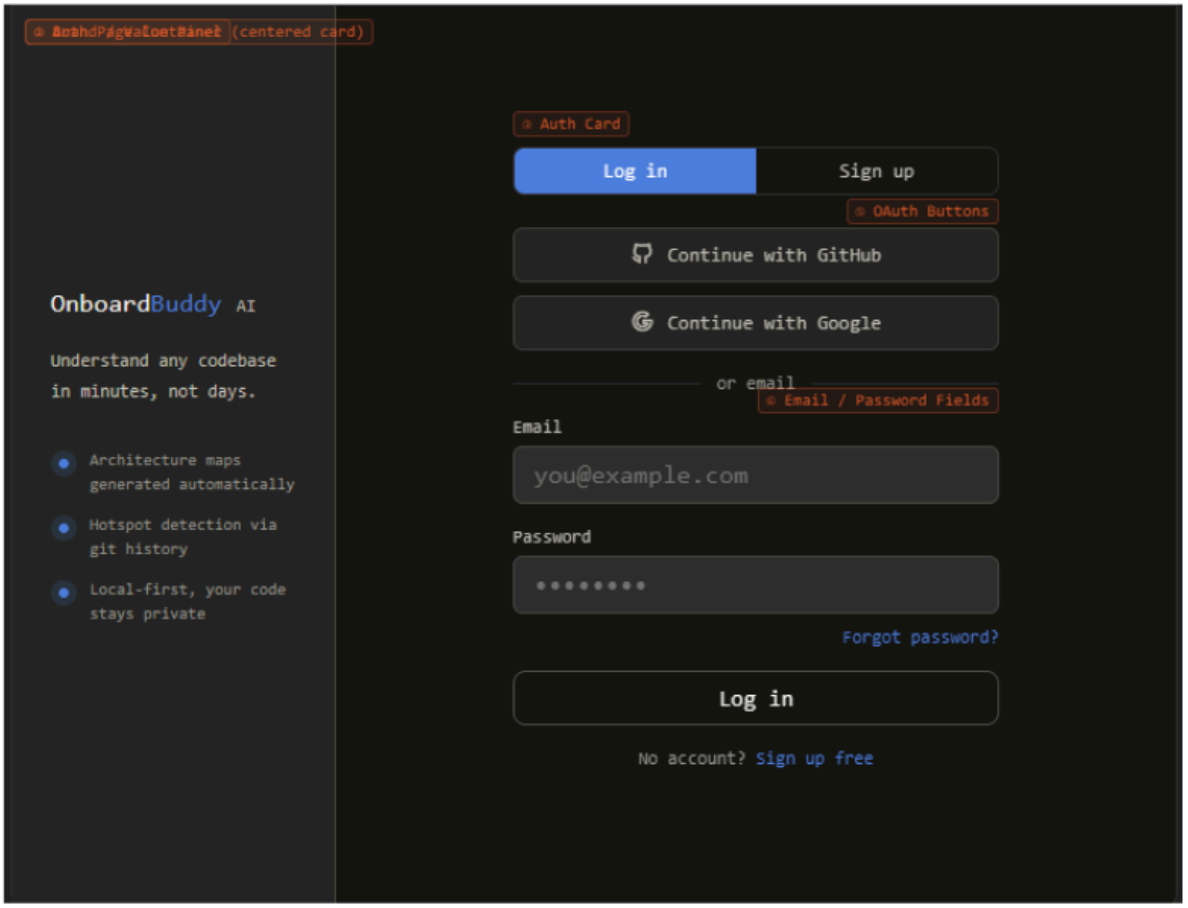
5) Wireframes

Note: These are “sketch-level” wireframes; replace with images later. Below are the required *labeled* wireframes and the narrative.

Wireframe 1 — Login/Signup Page

Purpose: to login or sign up to the web application
Elements:

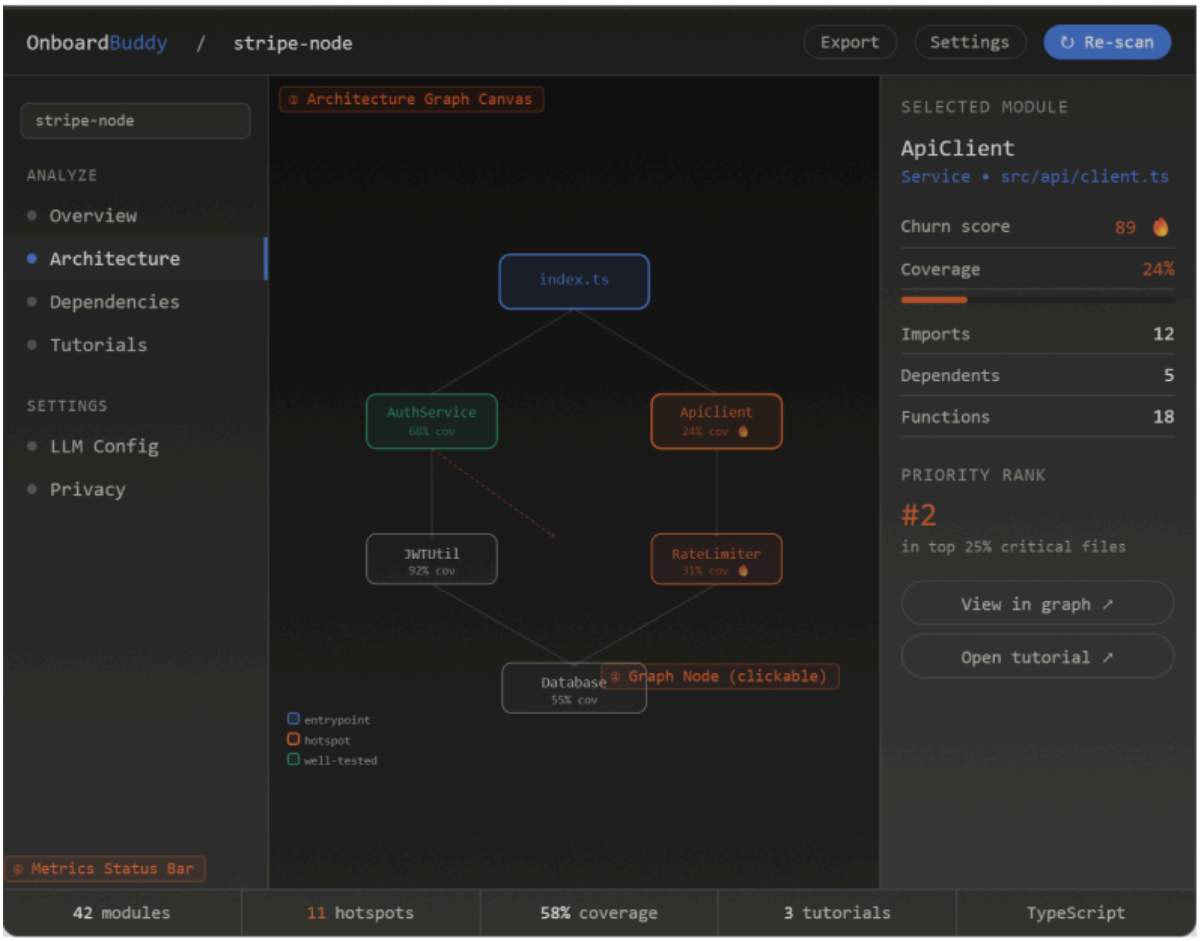
- GitHub and Google OAuth as primary sign-in options
- Email and password fields with "Forgot password?" link
- Tab toggle to switch between Log in and Sign up



Wireframe 2 — Architecture Map Dashboard (Main View)

Purpose: Visual overview of how the codebase is structured
Elements:

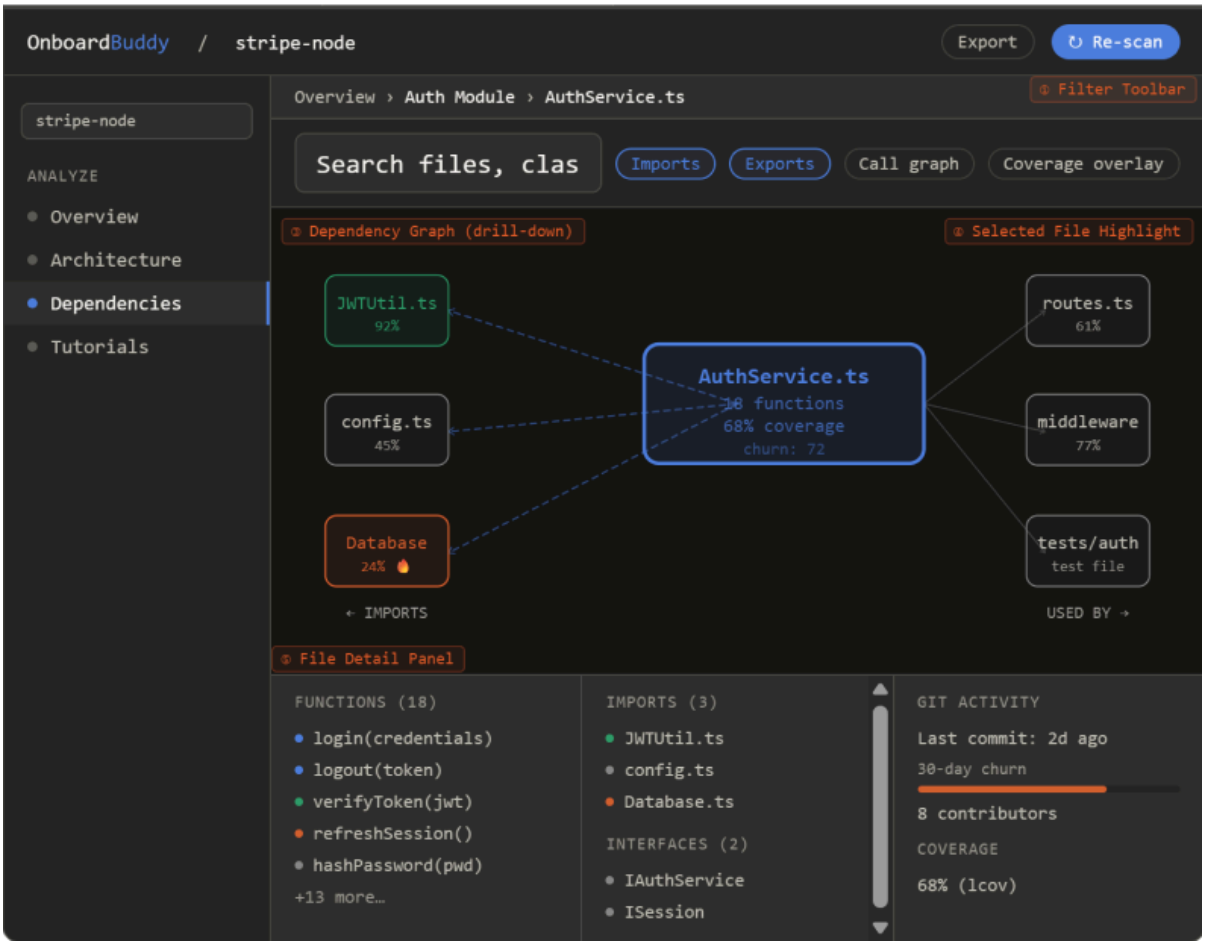
- Color-coded node graph (blue = entry, orange = hotspot, green = well-tested)
- Left sidebar for navigation between views
- Right inspector panel with churn score, coverage, and priority rank
- Bottom status bar with module count, hotspots, and coverage %



Wireframe 3 — Dependency Graph Explorer (Drill-down)

Purpose: Drill into a single file's relationships
Elements:

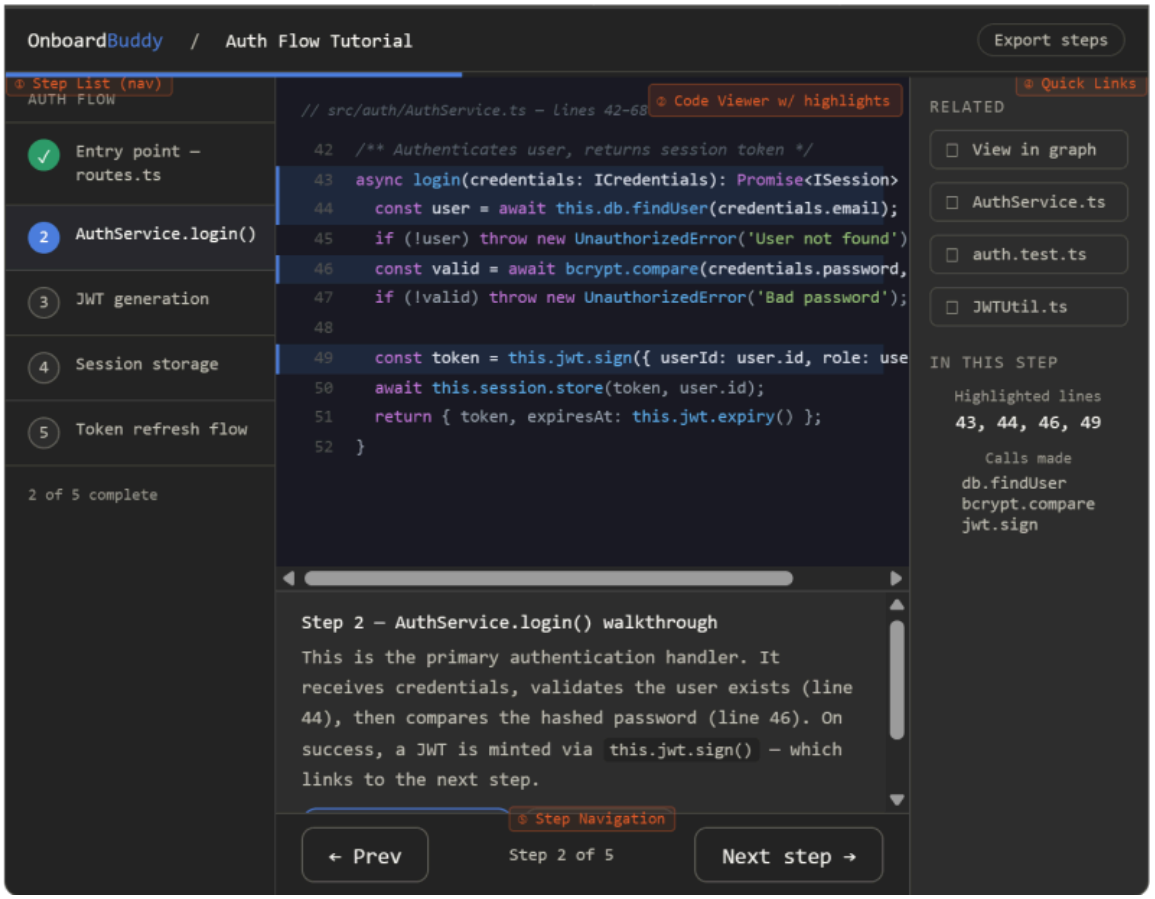
- Breadcrumb showing current drill-down path
- Filter chips for Imports, Exports, Call graph, Coverage overlay
- Radial graph with selected file centered, imports left, dependents right
- Bottom panel with function list, git churn, and coverage per file



Wireframe 4 — Interactive Tutorial View

Purpose: Step-by-step walkthrough of a key code workflow.
Elements:

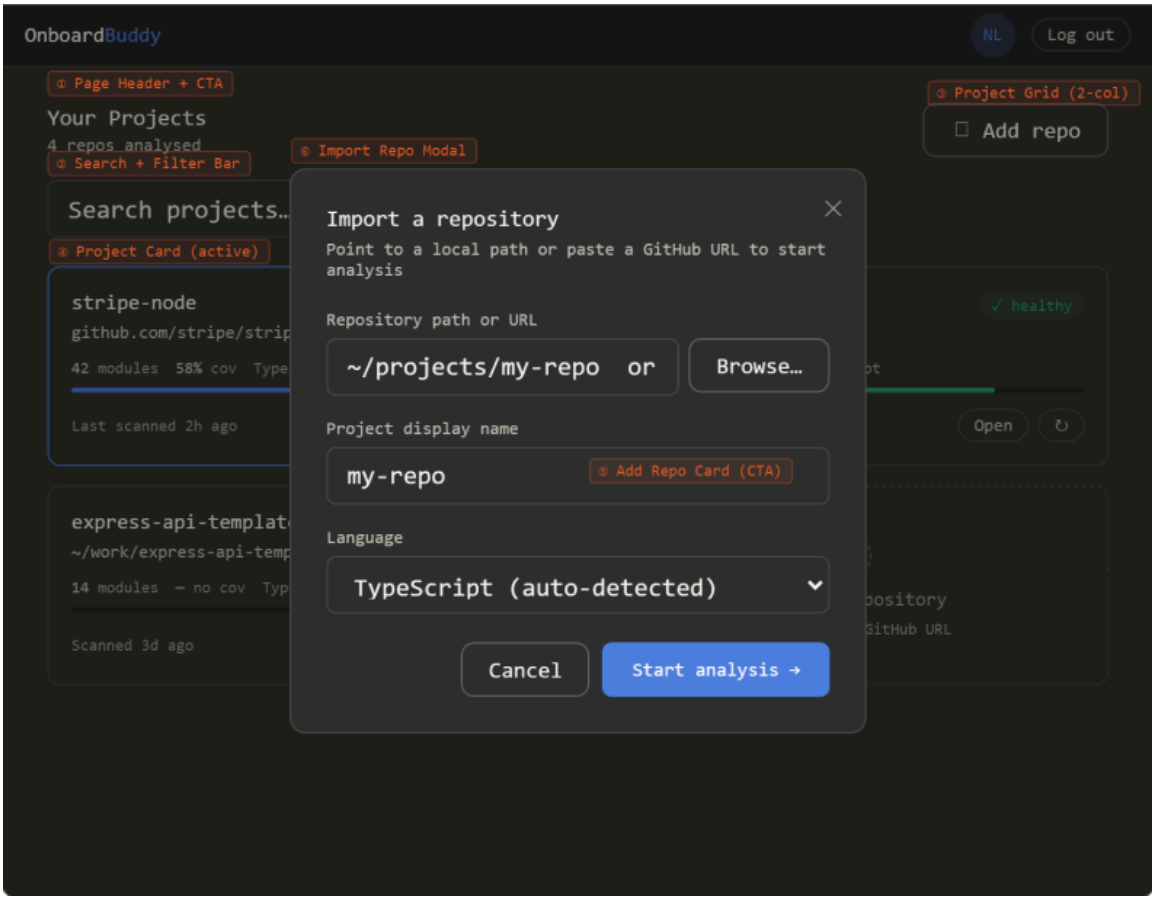
- Left step list with done / current / to-do states
- Dark code viewer with highlighted lines for the current step
- Explanation panel with prose, hotspot tags, and file:line reference
- Prev / Next navigation with a progress bar at the top



Wireframe 5 — Project List / Import Repo

Purpose: Home screen to view and manage all repos.
Elements:

- Project cards with name, coverage bar, health badge, and last-scanned time
- Search, sort, and filter bar
- Dashed "Import repo" empty-state card
- Import modal with path input, display name, and language selector

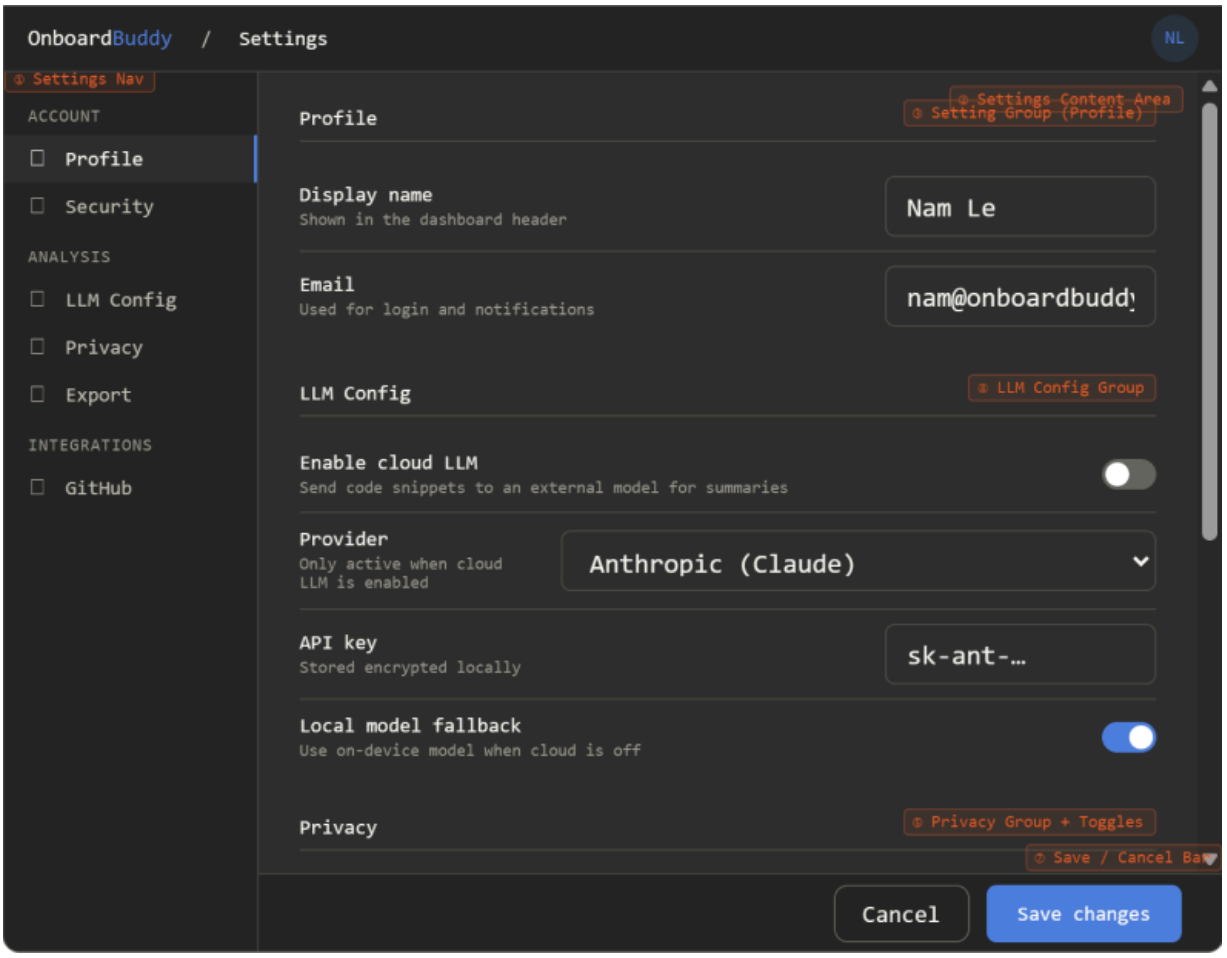


Wireframe 6 — Settings

Purpose: Control account, LLM provider, and privacy preferences

Elements:

- Left nav grouped into Account, Analysis, and Integrations
- LLM Config section with cloud toggle, provider selector, and API key field
- Privacy section with toggles
- Danger zone for deleting snapshots or account
- Sticky Save / Cancel bar at the bottom



6) Testing Plans

6.1 Frontend Test Plan

Tools: Jest/Vitest + React Testing Library (and/or Playwright for E2E)
Critical tasks to test

1. **Auth flow**
    - Login success/failure states
    - Token/session persistence
2. **Project selection**
    - Load project list
    - Handle empty state and error state
3. **Graph rendering**
    - Graph loads with given nodes/edges
    - Filtering (coverage/churn) changes view correctly
    - Node click opens details panel
4. **Tutorial walkthrough**
    - Step navigation (next/back)
    - Correct rendering of file references and step content
5. **Export/download**
    - Clicking export triggers download request and handles errors

Test cases / required data

- Mock API responses with:
    - Small graph (10 nodes) and medium graph (200 nodes)
    - Snapshot with missing coverage data
    - Snapshot with partial parsing errors (to test resilience)

6.2 Backend Test Plan

Tools: Mocha/Chai for assertions, Supertest for HTTP, mongodb-memory-server for in-memory DB
Critical tasks to test

1. **Auth & User**
    - **Unit test**
        i. Signup/login, password hashing
        ii. JWT signing, token rejection
    - **Integration test**
        i. creates user and returns token, rejects duplicate emails
        ii. return JWT on success, wrong password
2. **Project CRUD**
    - Create/list/get/delete projects, permissions per user
3. **Snapshot ingestion**
    - Upload analysis bundle validation (schema checks)
    - Versioning: multiple snapshots per project
4. **Analysis pipeline (unit tests)**
    - TS parsing on representative TS/TSX samples

    - Graph builder correctness on known fixtures
    - Coverage ingestion mapping correctness (file path normalization)

Test data

- Fixture repositories (small synthetic repos)
- Realistic sample repo snapshot (sanitized)
- Coverage fixture (lcov)
- Git history fixture (either real repo in test or mocked `git log` outputs)
- Use CPSC 310 project as test project as uses similar Tech Stack

Appendix: “Definition of Done” (MVP)

- Web app can analyze a TypeScript repo and produce:
    - Import graph/class dependency graph + basic call relationships (best-effort)
    - Coverage overlay + critical paths identification/class ranking (for architecture map)
    - 2–3 generated tutorials (request flow, auth flow, build/test flow)
- Dashboard can:
    - Authenticate user
    - Show project -> latest snapshot
    - Render graphs + tutorial steps
    - Export artifacts